Cross-Validation Techniques: A Comparative Study

Using the Cleveland Heart Disease Dataset

Nandini Bhatia, Diego Alberto, Khushpreet Kaur, Haruka Sawakami

July 27, 2026

Contents

1 Introduction

When evaluating predictive models, evaluating performance on the same dataset used for training often leads to overly optimistic performance estimates due to **overfitting**. Cross-validation refers to a set of statistical methods used to assess how accurately a predictive model will perform on new, unseen test data sets.

The primary goals of cross-validation are:

1. **Model Evaluation:** To obtain a reliable, unbiased estimate of model generalization performance.
2. **Model Selection:** To compare different modeling approaches or parameter settings to identify the optimal model.

In this project, we explore and compare four major cross-validation techniques using R:

- **Data Split Technique** (the one we covered in class)
- **Leave-One-Out Cross-Validation (LOOCV)**
- **k-fold Cross-Validation**
- **Repeated k-fold Cross-Validation**

By applying these methods to a practical data set, we aim to document their underlying concepts, demonstrate their implementation in R, and analyze the advantages and drawbacks regarding computational cost, bias, and variance to determine when to use each technique appropriately.

1.1 Model Performance Metrics

To evaluate and compare the predictive accuracy of our models across four different Cross-validation techniques, we utilize three standard continuous performance metrics:

1. **R-Squared (R^2):** Represents the squared correlation between the observed outcome values and the predicted values by the model. It ranges from 0 to 1 (or 0% to 100%). **Higher values indicate a better fit and stronger predictive power, meaning the better the model.**
2. **Root Mean Squared Error (RMSE):** Measures the average magnitude of the prediction error made by the model in predicting the outcome for an observation, giving higher weight to large errors due to squaring. **Lower values indicate better predictive accuracy, meaning the better the model.**
3. **Mean Absolute Error (MAE):** An alternative to the RMSE that is less sensitive to outliers. Measures the average absolute difference between predicted and observed values, treating all individual errors equally. **Lower values indicate better predictive accuracy, meaning the better the model.**

2 The Dataset

We selected the UCI Machine Learning Repository (cleveland.csv) to evaluate the four Cross-validation techniques based on the following criteria:

1. **Moderate Sample Size ($N = 303$)** This sample size ($N = 303$) is ideal for evaluating each technique. It is neither too small nor too large. It is large enough to retain sufficient data after dividing into two sets (especially for the Data Split technique), while being small enough to run computationally intensive methods like LOOCV efficiently without taking too much time.
2. **Reliability of Data Source and Practical Context** The UCI Machine Learning Repository is a widely recognized and trusted data source. In addition, predicting the presence of heart disease provides a clear, highly practical medical analytics case study for comparing performance across techniques.

2.1 Data Cleaning Pipeline

2.1.1 Step 1: Load the raw data and add column headers

```
# The file downloaded from UCI has NO header row, so we set header = FALSE
# and assign the official column names ourselves. We also tell R to treat
# both "?" and blank cells as missing values (na.strings).
raw <- read.csv("data/cleveland.csv", header = FALSE, na.strings = c("?", ""))

colnames(raw) <- c("age", "sex", "cp", "trestbps", "chol", "fbs", "restecg",
                   "thalach", "exang", "oldpeak", "slope", "ca", "thal", "num")

cat("Raw data dimensions:", nrow(raw), "rows x", ncol(raw), "columns\n")
```

```
## Raw data dimensions: 303 rows x 14 columns
```

```
head(raw)
```

```
##   age sex cp trestbps chol fbs restecg thalach exang oldpeak slope ca thal num
## 1  63   1  1      145  233   1       2     150     0     2.3     3  0    6   0
## 2  67   1  4      160  286   0       2     108     1     1.5     2  3    3   2
## 3  67   1  4      120  229   0       2     129     1     2.6     2  2    7   1
## 4  37   1  3      130  250   0       0     187     0     3.5     3  0    3   0
## 5  41   0  2      130  204   0       2     172     0     1.4     1  0    3   0
## 6  56   1  2      120  236   0       0     178     0     0.8     1  0    3   0
```

2.1.2 Step 2: Handle missing values

```
# Only 'ca' and 'thal' contain missing values in this dataset
cat("Missing values per column:\n")
```

```
## Missing values per column:
```

```r
print(colSums(is.na(raw)))
```

```
##      age      sex       cp trestbps     chol      fbs  restecg  thalach
##        0        0        0        0        0        0        0        0
##    exang  oldpeak    slope       ca     thal      num
##        0        0        0        4        2        0
```

```r
# Only 6 of 303 rows are affected (~2%), so we drop those rows rather
# than impute, to keep things simple
raw <- na.omit(raw)
cat("\nRows remaining after removing missing values:", nrow(raw), "\n")
```

```
##
## Rows remaining after removing missing values: 297
```

2.1.3 Step 3: Prepare the target variable

```r
# 'num' is coded 0-4 (0 = no disease, 1-4 = increasing severity).
# We binarize it: 0 = no disease, 1 = disease present
raw$target <- ifelse(raw$num > 0, 1, 0)
raw$num <- NULL

# Turn sex into labeled factors for readability, and keep target as it is
raw$sex    <- factor(raw$sex,    levels = c(0, 1), labels = c("female", "male"))
#raw$target <- factor(raw$target, levels = c(0, 1), labels = c("no_disease", "disease")

# Also turn other categorical variables into factors
categorical_cols <- c("cp", "fbs", "restecg", "exang", "slope", "ca", "thal")
raw[categorical_cols] <- lapply(raw[categorical_cols], factor)

heart <- raw
```

2.2 Exploratory Data Analysis

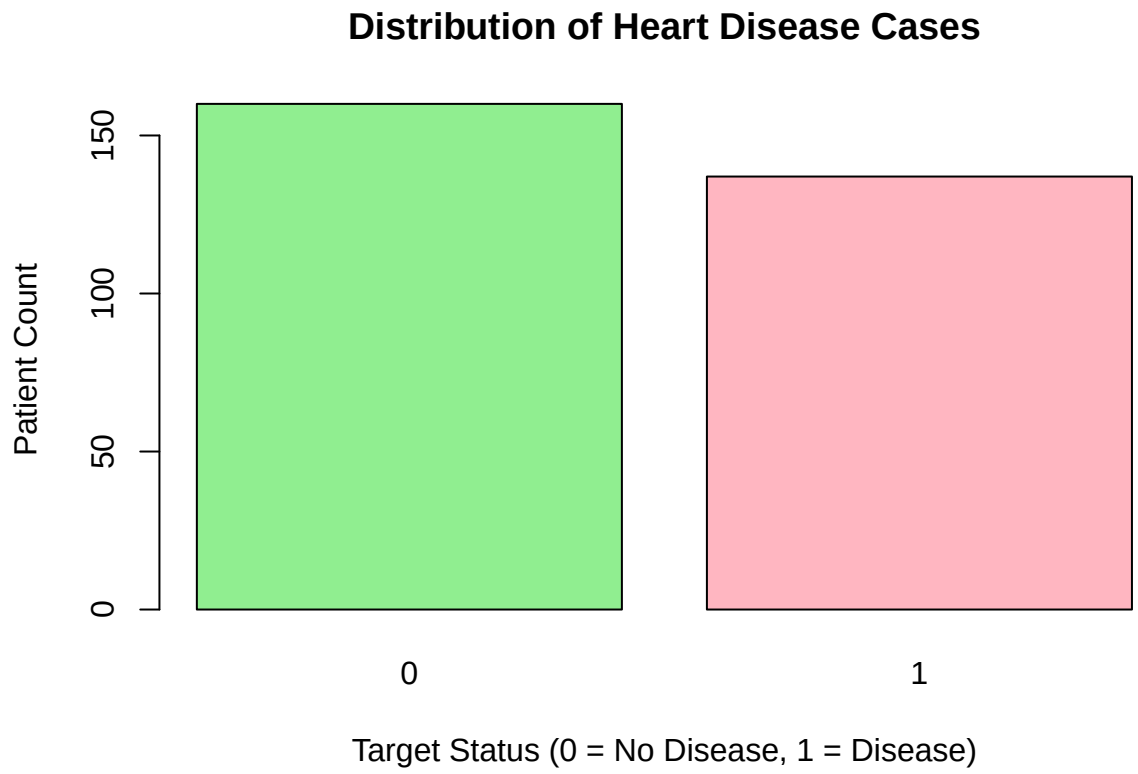
Before fitting predictive models, we examine the data set summary and key distribution characteristics of the variables.

```r
# Summary statistics for all variables
summary(heart)
```

```
##       age            sex      cp         trestbps          chol       fbs
##  Min.   :29.00   female: 96   1: 23   Min.   : 94.0   Min.   :126.0   0:254
##  1st Qu.:48.00   male  :201   2: 49   1st Qu.:120.0   1st Qu.:211.0   1: 43
##  Median :56.00                3: 83   Median :130.0   Median :243.0
##  Mean   :54.54                4:142   Mean   :131.7   Mean   :247.4
##  3rd Qu.:61.00                        3rd Qu.:140.0   3rd Qu.:276.0
##  Max.   :77.00                        Max.   :200.0   Max.   :564.0
##  restecg    thalach      exang      oldpeak      slope   ca      thal
##  0:147   Min.   : 71.0   0:200   Min.   :0.000   1:139   0:174   3:164
```

```
##  1:  4   1st Qu.:133.0   1: 97   1st Qu.:0.000   2:137   1: 65   6: 18
##  2:146   Median :153.0           Median :0.800   3: 21   2: 38   7:115
##          Mean   :149.6           Mean   :1.056           3: 20
##          3rd Qu.:166.0           3rd Qu.:1.600
##          Max.   :202.0           Max.   :6.200
##      target
##  Min.   :0.0000
##  1st Qu.:0.0000
##  Median :0.0000
##  Mean   :0.4613
##  3rd Qu.:1.0000
##  Max.   :1.0000
```

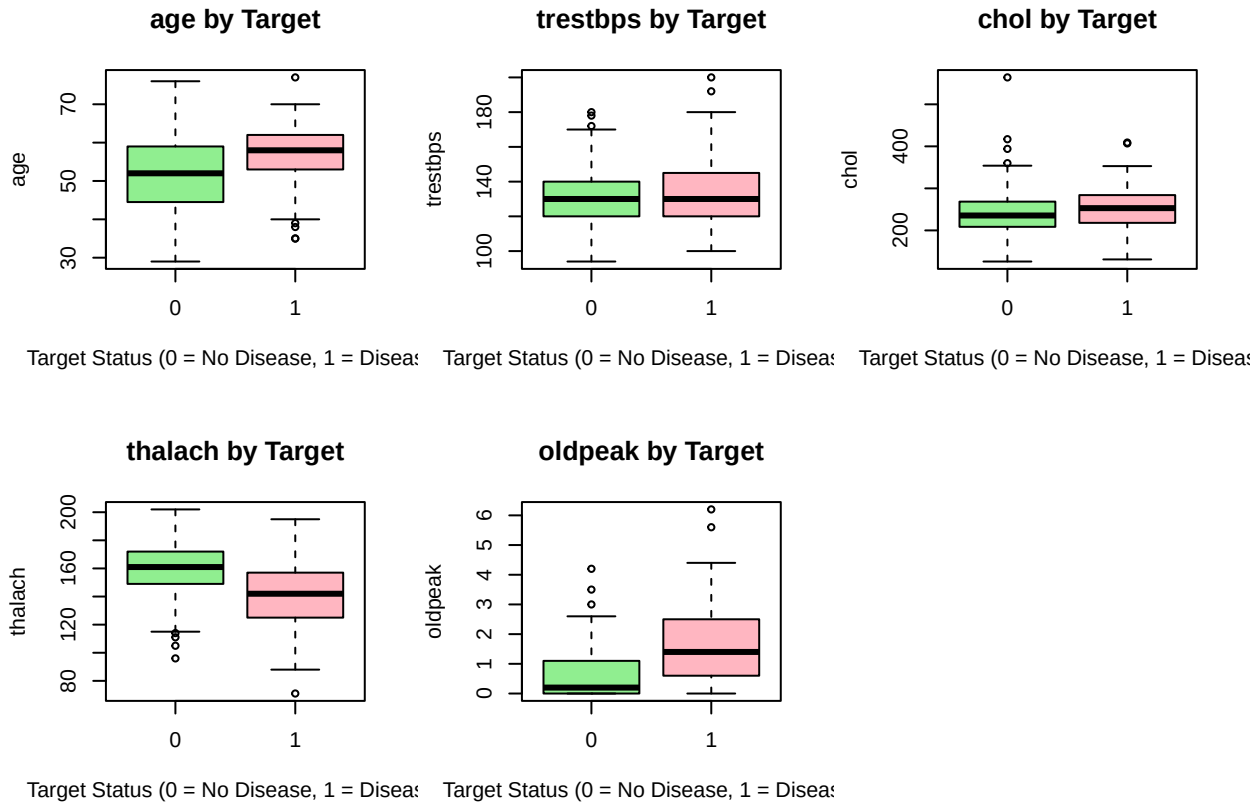
```r
# Simple barplot for target distribution
target_counts <- table(heart$target)
barplot(target_counts,
        main = "Distribution of Heart Disease Cases",
        xlab = "Target Status (0 = No Disease, 1 = Disease)",
        ylab = "Patient Count",
        col  = c("lightgreen", "lightpink"))
```




```r
# List of continuous numerical variables
num_vars <- c("age", "trestbps", "chol", "thalach", "oldpeak")

# Loop through each numerical variable and plot against target
```

```r
par(mfrow = c(2, 3))
for (var in num_vars) {
  boxplot(heart[[var]] ~ heart$target,
          main = paste(var, "by Target"),
          xlab = "Target Status (0 = No Disease, 1 = Disease)",
          ylab = var,
          col  = c("lightgreen", "lightpink"))
}
par(mfrow = c(1, 1))
```



3 Technique 1: Data Split (Train/Test)

3.1 The idea

Data Split is the simplest and most commonly used validation technique. The main idea is to randomly divide the dataset into two non-overlapping subsets:

- **Training Set (e.g., 80%):** Used to fit and estimate the parameters of the predictive model.
- **Test Set (e.g., 20%):** Kept completely untouched during training and used exclusively to evaluate how well the model performs.

By testing the model on data it has never seen before, this technique helps detect and prevent **overfitting**—a scenario where a model performs exceptionally well on training data but poorly on real-world test data.

Note that, this technique is only useful when we have a large data set enough to be partitioned. There is a disadvantage that we build a model on a fraction of the dataset only. It may cause a leaving out some important information or leading to higher bias.

3.2 Implementation

```
# Split the data into two set: training and test
training.samples <- heart$target %>% createDataPartition(p = 0.8, list = FALSE)
train.data <- heart[training.samples, ]
test.data  <- heart[-training.samples, ]

# Build the model
model <- lm(target ~., data = train.data)

# Make predictions and compute the R2, RMSE, and MAE
predictions <- model %>% predict(test.data)
data.frame(R2   = R2(predictions, test.data$target),
           RMSE = RMSE(predictions, test.data$target),
           MAE  = MAE(predictions, test.data$target))
```

```
##          R2      RMSE       MAE
## 1 0.3701296 0.4234554 0.3263939
```

3.3 Comment

- R^2 (≈ 0.370): Indicates that our linear model explains approximately 37.0% of the variance in heart disease presence using the training predictors.
- RMSE (≈ 0.423) & MAE (≈ 0.326): On average, the model's predictions deviate from the true binary outcome (0 or 1) by about 0.326. The higher RMSE relative to MAE suggests the presence of some larger prediction errors on specific test samples.
- Because the evaluation relies on a single random split, the test results are highly sensitive to how the data was partitioned (high variance in evaluation performance with this Data Split technique). This instability illustrates why more robust techniques, such as LOOCV or k-fold, are necessary for moderate-sized data sets. We will evaluate them in the next section.

While we fitted a linear regression model (lm) and evaluated its performance using R^2, RMSE, and MAE to align with our course curriculum, this approach has inherent limitations when applied to a binary outcome (0 or 1). So a binary target variable would have been converted into a factor and modeled using logistic regression (glm) since a linear regression can produce predicted values outside the valid range of probabilities, such as values less than 0 or greater than 1 (e.g., -0.1 or 1.25). Therefore, in addition to continuous error metrics like RMSE, incorporating classification metrics, such as Accuracy (by applying a 0.5 threshold to convert continuous predictions into binary outcomes 0 or 1), might provide a more comprehensive and practically meaningful comparison of the models.

4 Technique 2: Leave-One-Out Cross-Validation (LOOCV)

4.1 The idea

Leave-One-Out Cross-Validation (LOOCV) is the most extreme version of k-fold cross-validation: for a data set with n observations, the model is fit n separate times. Each time, exactly **one** observation is left out to serve as the test, and the model is trained on the remaining $n - 1$ observations. The single observation is then predicted and compared to its true value. After repeating this for every observation in the dataset, the n individual error estimates are averaged to produce the final LOOCV performance estimate.

Because LOOCV is a special case of k-fold CV where $k = n$, it uses almost all of the data for training in every iteration, which tends to give a low-bias estimate of model performance. The trade-off is computational cost: for our data set of 297 rows, LOOCV requires fitting the linear model 297 separate times, compared to just 1 fit for the data split technique in Technique 1.

4.2 Implementation

```
# Make sure target is numeric (not a factor) so lm can run a regression
if (is.factor(heart$target)) {
  heart$target <- ifelse(heart$target == "disease", 1, 0)
}

# Training control: LOOCV
train.control <- trainControl(method = "LOOCV")

# Train the same model as in Technique 1, but with LOOCV this time
loocv_model <- train(target ~ ., data = heart, method = "lm",
                     trControl = train.control)

# Show the R2, RMSE, and MAE
print(loocv_model)
```

```
## Linear Regression
##
## 297 samples
##  13 predictor
##
## No pre-processing
## Resampling: Leave-One-Out Cross-Validation
## Summary of sample sizes: 296, 296, 296, 296, 296, 296, ...
## Resampling results:
##
##   RMSE       Rsquared   MAE
##   0.3518281  0.5039159  0.2728333
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

4.3 Comment

- R^2 (≈ 0.504): Indicates that our linear model explains approximately 50.4% of the variance in heart disease presence using the predictors. The remaining variation is likely driven by factors not captured in this dataset.

- RMSE (≈ 0.352) & MAE (≈ 0.273): On average, the model's predictions deviate from the true binary outcome (0 or 1) by about 0.273. The noticeable gap between RMSE and MAE suggests the errors are not uniform across patients, a subset of patients are predicted considerably worse than the "average" patient, which pulls RMSE upward relative to MAE.
- Because LOOCV averages the error across all 297 individual held-out predictions (every patient gets exactly one turn as the test case), this estimate is not sensitive to how the data happens to be split. This makes the reported R^2 and RMSE/MAE a more reliable estimate of how the model would perform on a genuinely new patient than a single train/test split would provide.

5 Technique 3: k-fold Cross-Validation

5.1 The idea

k-fold Cross-Validation represents a highly effective middle ground between the simple Data Split and the computationally expensive LOOCV. The main idea is to randomly divide the dataset into k roughly equal-sized groups, or "folds" (typically, $k = 5$ or $k = 10$).

The model training and evaluation process is then repeated exactly k times. In each iteration:

- **One fold** is held out and treated as the test set.
- The **remaining** $k - 1$ **folds** are combined and used as the training set to fit the model.
- The model's performance is calculated on the single held-out fold.

This process ensures that every single observation in the dataset gets to be in the test set exactly once. The final performance metrics are obtained by averaging the k individual error estimates. k-fold CV is far less computationally intensive than LOOCV (requiring only k fits instead of n fits) while still producing a much more stable and reliable estimate of model performance than a single Data Split, successfully balancing the bias-variance trade-off.

5.2 Implementation

```
# Define the training control: 10-fold Cross-Validation
train.control.kfold <- trainControl(method = "cv", number = 10)
# Train the same linear model, but using 10-fold CV this time
set.seed(2026) # Set seed for reproducibility of the random folds
kfold_model <- train(target ~ ., data = heart, method = "lm",
                     trControl = train.control.kfold)

kfold_model$resample %>% summarise(sd_RMSE = sd(RMSE), sd_Rsquared = sd(Rsquared))
```

```
##      sd_RMSE sd_Rsquared
## 1 0.02770891  0.07525962
```

```
# Show the R2, RMSE, and MAE
print(kfold_model)
```

```
## Linear Regression
##
## 297 samples
##  13 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 267, 267, 268, 267, 267, 267, ...
## Resampling results:
##
##   RMSE       Rsquared   MAE
##   0.3515187  0.4986169  0.2715469
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

5.3 Comment

- R^2 (≈ 0.499): Indicates that our linear model explains approximately 49.9% of the variance in the presence of heart disease. Interestingly, this R^2 is slightly higher than both the single Data Split (0.370) and LOOCV (0.504), suggesting that averaging across 10 random folds provided a highly stable estimate of the model's true predictive capability.
- RMSE (≈ 0.352) & MAE (≈ 0.272): On average, the model's predictions deviate by about 0.272. Both of these error metrics are slightly lower (better) than those seen in the Data Split and LOOCV techniques.
- Bias-Variance Trade-off: 10-fold CV strikes an excellent balance. It avoids the high variance associated with a single train/test split (because performance is averaged over 10 tests), and it largely avoids the high variance sometimes associated with LOOCV (since training sets in LOOCV are nearly identical to each other, their error estimates can be highly correlated). Furthermore, it is significantly faster to compute, making k-fold CV the industry standard for most practical machine learning workflows.

6 Technique 4: Repeated k-fold Cross-Validation

6.1 The idea

Repeated k-fold Cross-Validation builds directly on Technique 3. Plain k-fold CV already averages performance over k folds instead of relying on a single train/test split, but with a moderate dataset like ours (only 297 rows split into 10 folds of roughly 30 patients each), which specific patients happen to land in which fold can still nudge a single k-fold run's result up or down somewhat.

Repeated k-fold CV addresses this remaining source of variance by running the entire k-fold procedure multiple times, each time with a completely new random partition of the data into folds, and then averaging the results across all repeats. For example, "10-fold CV repeated 5 times" means we perform 5 independent 10-fold CV runs (50 total model fits in total) and average the 5 resulting performance estimates together.

Repeating the process does not reduce bias any further compared to plain k-fold (bias is governed by k, i.e., how much data is used for training in each fold), but it does reduce the variance of the performance estimate, since we are no longer relying on the luck of one particular fold assignment. The trade-off is computational cost: as shown in the cost table below, repeating 10-fold CV 5 times requires 50 model fits instead of just 10.

6.2 Implementation

```r
# Define the training control: 10-fold CV repeated 5 times
train.control.repeated <- trainControl(method = "repeatedcv", number = 10, repeats = 5)

# Train the same linear model, but using repeated 10-fold CV this time
set.seed(2026) # Set seed for reproducibility of the random folds across all 5 repeats
repeated_cv_model <- train(target ~ ., data = heart, method = "lm",
                           trControl = train.control.repeated)

# Show the R2, RMSE, and MAE
print(repeated_cv_model)
```

```
## Linear Regression
##
## 297 samples
##  13 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold, repeated 5 times)
## Summary of sample sizes: 267, 267, 268, 267, 267, 267, ...
## Resampling results:
##
##   RMSE       Rsquared   MAE
##   0.3527488  0.5124759  0.2748135
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

```r
# Look at the spread across all 50 individual resamples, to compare
# against the single 10-fold run's variability in Technique 3
repeated_cv_model$resample %>%
  summarise(sd_RMSE = sd(RMSE), sd_Rsquared = sd(Rsquared))
```

```
##      sd_RMSE sd_Rsquared
## 1 0.04629221   0.1186108
```

6.3 Comment

- R^2 (≈ 0.512): Indicates that our linear model explains approximately 51.2% of the variance in heart disease presence, averaged over all 5 repeats of 10-fold CV. This is modestly higher than both the single 10-fold estimate from Technique 3 ($R^2 \approx 0.499$) and LOOCV ($R^2 \approx 0.504$), consistent with repeating the procedure giving a more stable read on the model's true predictive capability rather than any one lucky/unlucky fold assignment.
- RMSE (≈ 0.353) & MAE (≈ 0.275): On average, the model's predictions deviate from the true binary outcome (0 or 1) by about 0.275. These are very close to the single 10-fold and LOOCV results, which makes sense: repeating k-fold does not change how much data each fold trains on, so it doesn't systematically shift RMSE or MAE up or down – it only changes how precisely we've estimated them.
- Stability of the estimate: the raw spread across individual fold scores (`sd_RMSE`) was actually higher here (0.0463 across the 50 resamples) than in the single 10-fold run (0.0277 across 10 resamples) – pooling fold scores from 5 different random partitions naturally shows more raw variety than one

partition does. But what determines how precisely we know the *average* RMSE is the standard error of that mean ($\text{sd}/\sqrt{n_{\text{resamples}}}$), and this is where repeating pays off: the standard error dropped from $0.0277/\sqrt{10} \approx 0.0088$ for the single 10-fold run to $0.0463/\sqrt{50} \approx 0.0065$ for the repeated version – roughly a 26% tightening of our estimate, achieved purely by re-shuffling the folds and re-running the same 10-fold procedure 5 times.

7 When to Use Which Technique

Beyond simply implementing each technique, it is important to understand *when* each one is the right tool for the job. This section reflects on the four techniques we have implemented so far, Data Split, LOOCV, and k-fold CV, and Repeated k-fold CV, using both the theory behind them and what we actually observed on the Cleveland Heart Disease dataset.

7.1 Dataset size matters most

- With the **Data Split** technique, holding out 20% for testing left only about 59 patients in the test set. A test set this small means a single unlucky (or lucky) split can swing R^2 and RMSE noticeably – which is consistent with what we found: Data Split gave the lowest R^2 (≈ 0.370) of the four techniques, and this number would likely shift if a different random seed were used.
- **LOOCV**, **k-fold CV**, and **Repeated k-fold CV** all use nearly all of the data for training in every iteration, which is why they gave higher and more consistent estimates ($R^2 \approx 0.504$, 0.499, and 0.512 respectively). For a dataset much larger than ours (tens of thousands of rows), a single data split would be far less risky, since even a 20% test set would still contain thousands of observations.
- With only 297 rows split into 10 folds, each fold contains roughly 30 patients – small enough that *which* specific patients land in which fold can still shift a single k-fold run's result somewhat. **Repeated k-fold CV** addresses exactly this by re-shuffling the folds and re-running k-fold 5 times (50 total fits), averaging out that remaining randomness. We can see this directly in our results: the standard error of the RMSE estimate dropped from **0.0088** with a single 10-fold run to **0.0065** with the repeated version, a roughly 26% reduction in the uncertainty of our performance estimate, achieved simply by repeating the same procedure on different random fold partitions. This matters less as dataset size grows, since larger folds are less sensitive to which specific observations they contain.

Rule of thumb: the smaller the dataset, the more a single data split should be avoided in favor of LOOCV, k-fold, or repeated k-fold CV – and the more repetitions of k-fold are worth their extra computational cost.

7.2 Computational Cost

This is where the four techniques differ the most in practice, even though our dataset is small enough that all four ran quickly:

Technique	Model fits required (n = 297, k = 10, repeats = 5)
Data Split	1
LOOCV	297
k-fold CV (k=10)	10
Repeated k-fold CV	50

For a simple linear model like ours, fitting the model up to 297 times is still fast (all four techniques ran in well under a second combined). But if the model were more expensive to train (e.g., a random forest with many trees, or a neural network), LOOCV's cost would become a real constraint, and even repeated k-fold's

50 fits would need to be scaled back (fewer repeats, or a smaller k), while a single Data Split would remain the cheapest option by far.

7.3 Bias vs. variance trade-off

Understanding the bias-variance trade-off is critical when selecting a model evaluation technique, as each method strikes a different balance between these two sources of error:

- **Bias:** Refers to the error introduced by approximating a real-world problem with a simplified model. High bias can cause a validation technique to under- or overestimate model performance systematically (e.g., training on too little data).
- **Variance:** Refers to the variability or sensitivity of the performance estimate when fitted on different random subsets or samples of the same dataset. High variance means the performance metric fluctuates significantly depending on how the data happens to be partitioned.

Here is how the four cross-validation techniques compare along the bias-variance spectrum:

1. **Data Split (High Variance, Potential High Bias):** Holding out 20% of our moderate-sized dataset ($N = 297$) for testing left only about 59 observations in the test set. Because the model was trained on only 80% of the available data, it suffers from **higher bias** (pessimistic evaluation), resulting in a lower R^2 (≈ 0.370) compared to the other methods. Furthermore, it suffers from **high variance** because the single test evaluation depends heavily on which specific 59 observations ended up in the test set.

2. **LOOCV (Low Bias, High Variance):** Because LOOCV trains the model on $n-1$ (296) observations in every iteration, it utilizes nearly 100% of the data, resulting in **virtually unbiased** performance estimates ($R^2 \approx 0.504$). However, because each of the 297 trained models uses an almost identical subset of the data (differing by only a single observation), the outputs across iterations are highly correlated. Averaging outputs from highly correlated models can lead to **higher variance** in the estimated test error across different samples of data.

3. **k-fold Cross-Validation (Balanced Bias and Variance):** k-fold CV ($k = 10$) strikes an optimal balance. By training on 90% of the data in each iteration, the bias remains very low—nearly as low as LOOCV. At the same time, because the training sets across folds overlap less than in LOOCV, the validation estimates are less correlated, significantly reducing the variance. On our dataset, 10-fold CV yielded a stable $R^2 \approx 0.499$ and RMSE ≈ 0.352.

4. **Repeated k-fold Cross-Validation (Lowest Variance of the Estimate):** While plain k-fold CV balances bias and variance well, a single run on a small/moderate dataset can still be influenced by how observations happen to be partitioned across the k folds. Repeating 10-fold CV 5 times (50 total resamples) does not change the level of bias — training still uses 90% of the data each fold, exactly as in plain k-fold. What it changes is how precisely we know the *average* performance: the raw spread across individual fold scores (sd_RMSE ≈ 0.0463 over 50 resamples) is naturally larger than the single 10-fold run's (sd_RMSE ≈ 0.0277 over 10 resamples), simply because it pools scores from 5 different random partitions rather than one. But the standard error of the mean RMSE — sd divided by $\sqrt{n_{\text{resamples}}}$ — dropped from ≈ 0.0088 (single run) to ≈ 0.0065 (repeated), a roughly 26% tightening of our estimate. This is the correct sense in which repeated k-fold offers the most reliable assessment: not a lower raw spread, but a more precise estimate of the same underlying bias.

7.4 Practical decision guide

Based on both the theory and what we observed on this dataset, the choice of technique comes down to three practical questions:

1. How much data do you have?
2. How expensive is the model to train?
3. How precise does the performance estimate need to be?

Our recommendation for this dataset: given its moderate size (297 rows) and the low cost of fitting a linear model, **Repeated 10-fold CV** is the most appropriate choice for reporting a final performance estimate — it uses nearly all the data for training in every fit (like k-fold and LOOCV), avoids LOOCV's correlated-model variance issue, and gives the most stable estimate of the four techniques at a computational cost (50 fits) that remains trivial for this model. Plain 10-fold CV remains a reasonable and much faster alternative if only a single, "good enough" estimate is needed rather than the most precise one.

8 References

Janosi A, Steinbrunn W, Pfisterer M, Detrano R. Heart Disease [dataset]. 1989. UCI Machine Learning Repository. Available from: https://doi.org/10.24432/C52P4X.

Cross-Validation in R: Validation Set, LOOCV & k-Fold with tidymodels. Modified July 7, 2026. Datanovia. Available from: https://www.datanovia.com/learn/machine-learning/model-validation/cross-validation